Flutter Interview Q&A Sheet (Freshers)

🧠 1. What is Flutter?

Answer:
Flutter is an *open-source UI toolkit* developed by Google used to build *cross-platform applications* from a single codebase.

It enables developers to create high-performance apps for *Android, iOS, web, and desktop* using the Dart language.

It uses a *reactive framework* and *widget-based architecture*.

⚙️ 2. What is Dart?

Answer:
Dart is a *client-optimized programming language* used by Flutter.

It supports:

- Object-Oriented Programming (OOP)
- Asynchronous programming (Future, async/await)

Dart is optimized for *UI development* and provides *fast performance with Ahead-of-Time (AOT) compilation*.

🧩 3. What are Widgets in Flutter?

Answer:
Widgets are the *building blocks of a Flutter application*.

Everything in Flutter is a widget, including:

- Layout (Row, Column)
- UI elements (Text, Button)

Widgets follow a *declarative UI approach*, meaning UI is rebuilt based on state changes.

🔄 4. Stateless vs Stateful Widget

Answer:

- **Stateless Widget**
  - Immutable (cannot change)
  - No internal state
  - Example: Text, Icon
- **Stateful Widget**
  - Mutable (can change during runtime)
  - Has state managed by `setState()`
  - Example: Form, Checkbox

👉 HR Keyword: *State management*

🔥 5. What is Hot Reload?

Answer:
Hot Reload allows developers to *instantly see code changes* without restarting the app.

It improves:

- Developer productivity
- Faster debugging

👉 HR Word: *Rapid development cycle*

🏗️ 6. Difference between Hot Reload and Hot Restart

Answer:

Feature	Hot Reload	Hot Restart
Preserves state	✅ Yes	❌ No
Speed	Fast	Slightly slower
Use case	UI changes	Major logic reset

📦 7. What is pubspec.yaml?

Answer:
It is the *configuration file* in Flutter used to manage:

- Dependencies
- Assets (images, fonts)
- Project metadata

👉 HR Word: *Dependency management*

🧠 8. What is BuildContext?

Answer:
BuildContext represents the *location of a widget in the widget tree*.

It is used to:

- Access theme data
- Navigate between screens
- Access inherited widgets

🧭 9. What is Navigator?

Answer:
Navigator is used for *routing and navigation* between screens.

Common methods:

- `Navigator.push()`
- `Navigator.pop()`

👉 HR Word: *Navigation stack management*

🔗 10. What is State Management?

Answer:
State management refers to how *data is handled and updated* in an app.

Common approaches:

- setState (basic)
- Provider
- Riverpod

- Bloc

👉 HR Word: *Scalable architecture*

📱 11. What is a Scaffold?

Answer:
Scaffold provides a *basic structure* for UI:

- AppBar
- Body
- FloatingActionButton

👉 It acts like a *layout skeleton*.

🎨 12. Difference between Container and SizedBox

Answer:

- **Container**
  - Flexible widget
  - Supports styling (color, padding, margin)
- **SizedBox**
  - Used for fixed spacing
  - Lightweight

⏱️ 13. What are Future and async/await?

Answer:
They are used for *asynchronous programming*.

- `Future` → represents a value available later
- `async/await` → simplifies async code

👉 Used in APIs, database calls.

🌐 14. How do you call an API in Flutter?

Answer:
Using packages like:

- http package

Steps:

1. Send request
2. Await response
3. Parse JSON

👉 HR Word: *API integration*

🧪 15. What is null safety in Dart?

Answer:
Null safety prevents *null-related runtime errors*.

It ensures variables cannot be null unless specified.

👉 Improves *code reliability and robustness*.

📐 16. What is Widget Tree?

Answer:
Flutter UI is structured as a *tree of widgets*.

- Parent widget
- Child widget

👉 HR Word: *Hierarchical structure*

⚡ 17. What is setState()?

Answer:
Used in Stateful widgets to *update UI when data changes*.

When called:
👉 Flutter rebuilds the widget

📊 18. What is ListView?

Answer:
ListView is used to display *scrollable lists of items*.

Types:

- ListView.builder (optimized for large data)

🎯 19. What is main() function in Flutter?

Answer:
Entry point of Flutter app:

```
void main() {
  runApp(MyApp());
}
```

🏁 20. What is runApp()?

Answer:
It initializes the Flutter app and attaches the *root widget* to the screen.

💡 Bonus HR-Level Questions (Flutter Specific)

❓ 21. How do you ensure performance optimization in Flutter?

Answer:

- Use const widgets
- Avoid unnecessary rebuilds
- Use ListView.builder
- Optimize images

👉 HR Words:

- *Performance optimization*
- *Efficient rendering*

❓ 22. What challenges have you faced in Flutter?

Answer:
“While working on Flutter projects, I faced challenges in *state management and UI responsiveness*.

I addressed them by exploring better solutions like Provider and optimizing layouts, which improved overall performance.”

❓ 23. How do you handle debugging?

Answer:
“I use:

- Debug console
- Flutter DevTools

I follow a *systematic debugging approach* to identify and resolve issues efficiently.”

More Questions-Answers [Here]